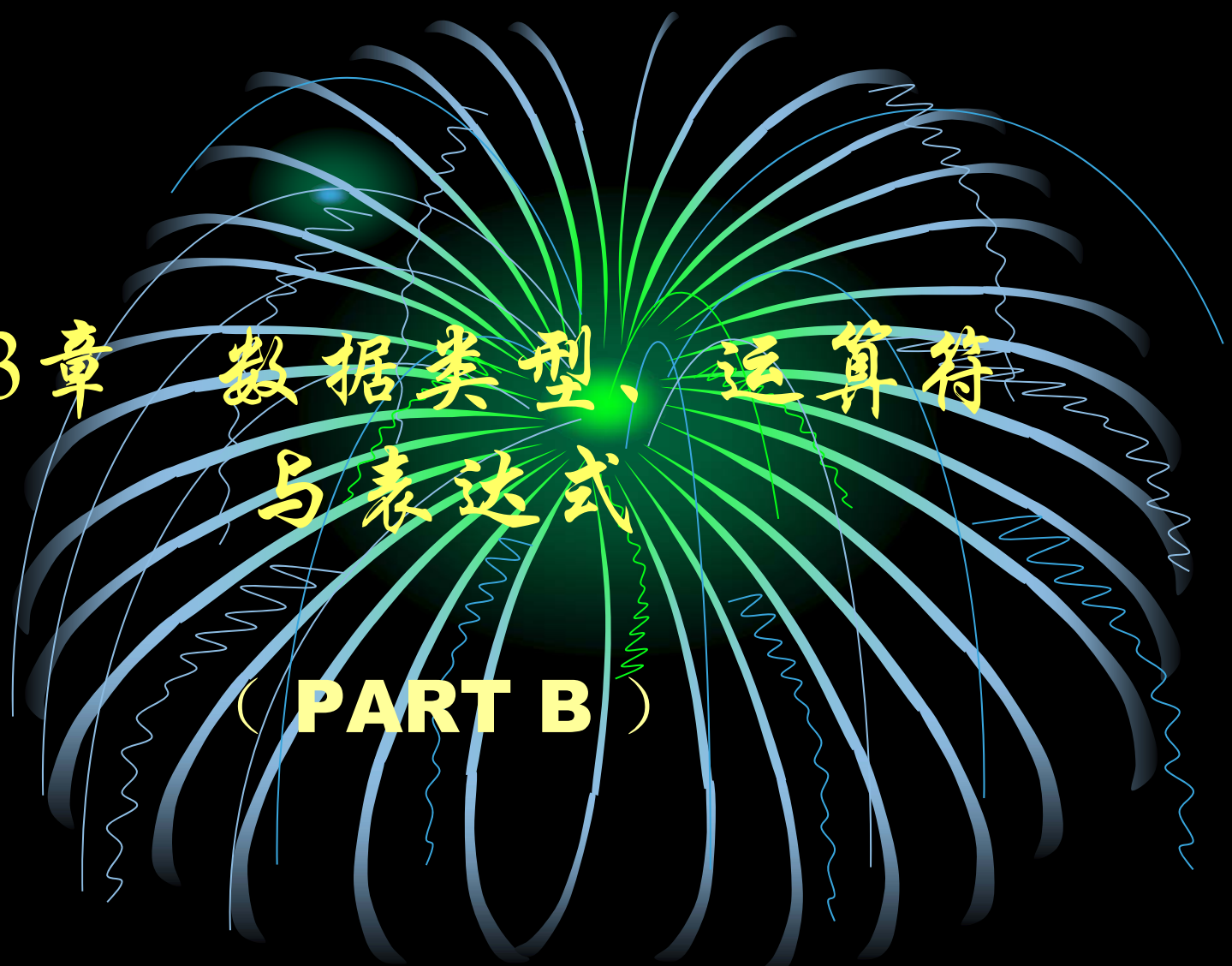
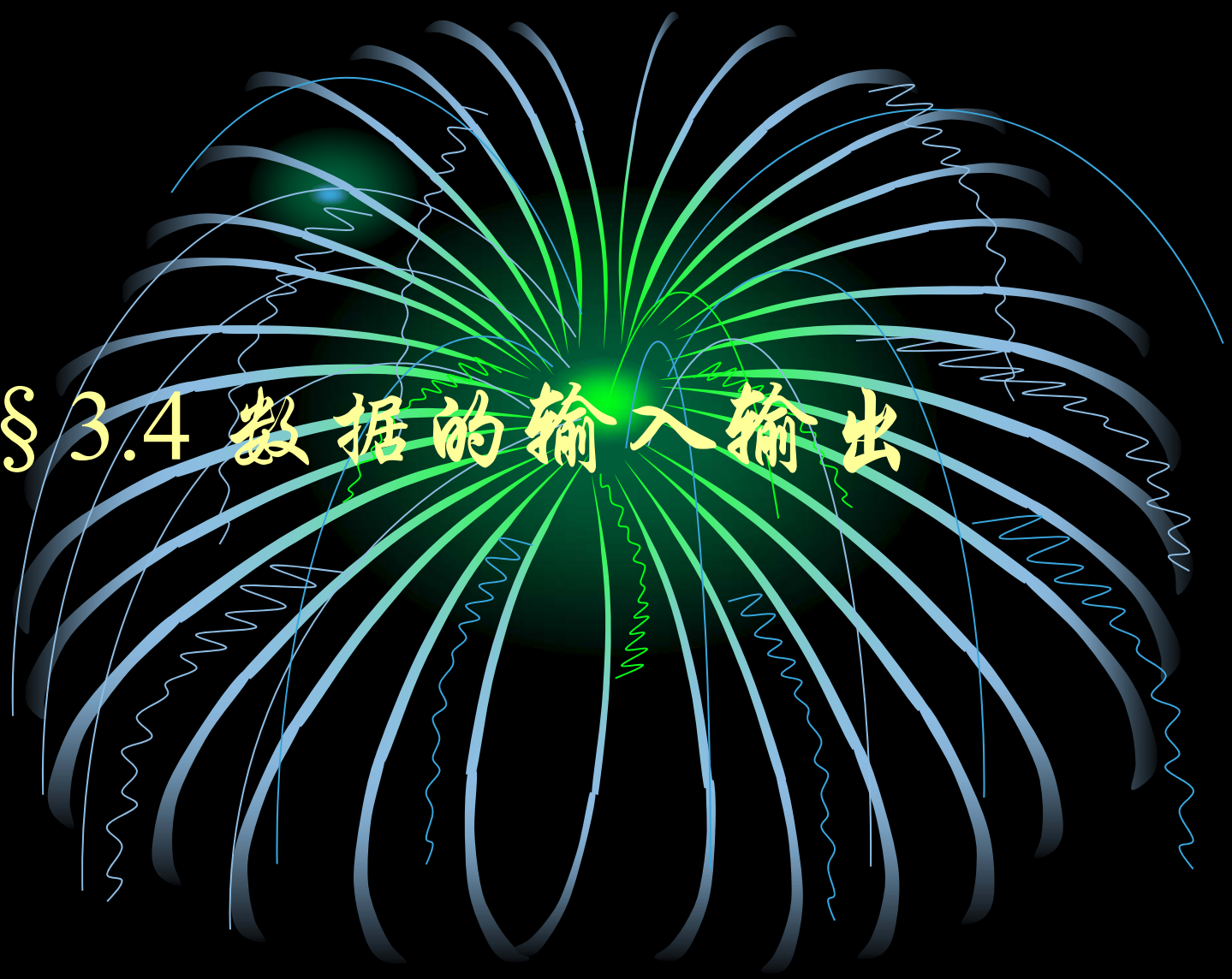




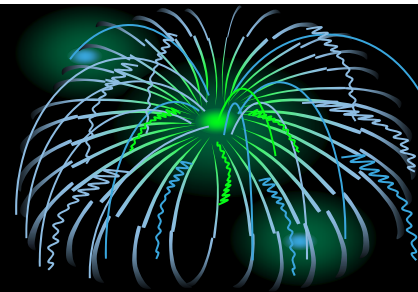
第3章 数据类型、运算符 与表达式

(PART B)



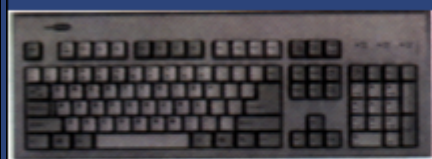
§ 3.4 数据的输入输出

数据输入输出的概念



- 输入输出是相对**计算机主机**而言的。
 - **输出** —— 从计算机向外部输出设备输出数据
 - **输入** —— 从外部输入设备输入数据

输入设备



键盘



鼠标



数码相机

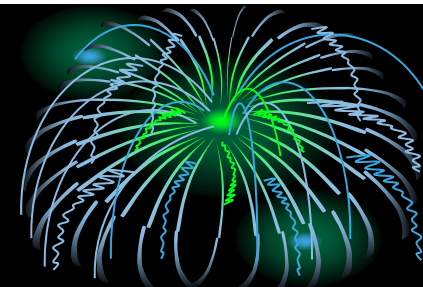


手写笔



摄像头

数据输入输出在C语言中的实现

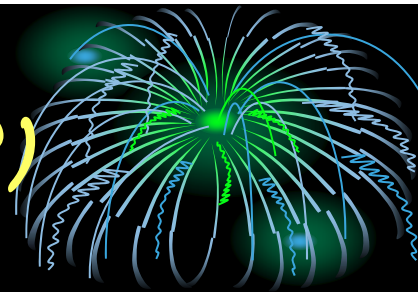


- **C语言本身不提供输入输出语句，而由C标准库函数库中的函数来实现相应操作！**
 - **printf**和**scanf**是C提供的库函数，不是输入输出语句！
 - C提供的标准函数以库的形式在C的编译系统中提供，它们不是C语言文本中的组成部分！

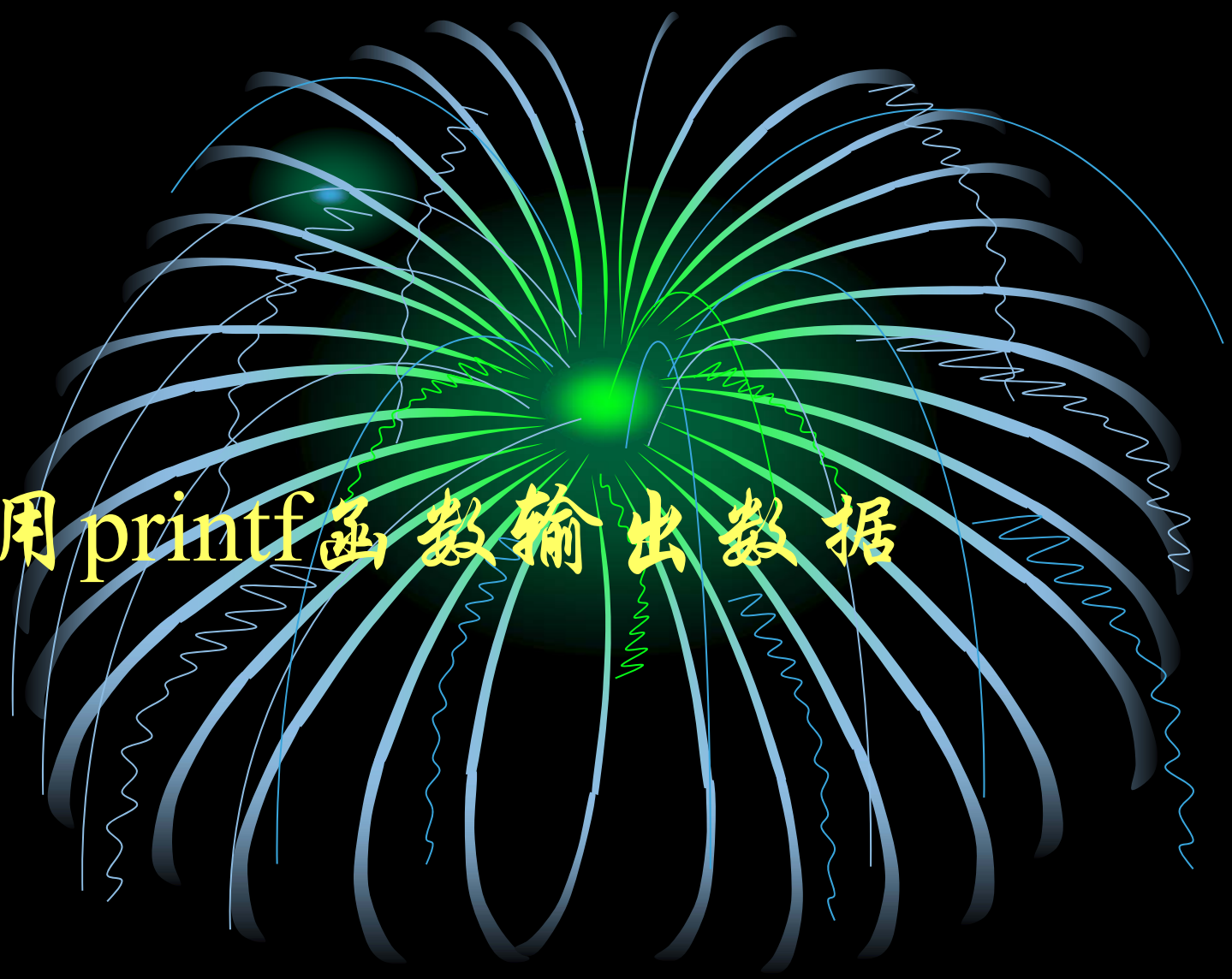
【目的】

- 1) 使C语言编译系统简化：避免在编译阶段处理与硬件有关的问题；
 - 2) 通用性强、可移植性好：只需针对不同的机器编写相应的函数库。
- **标准的输入输出函数**：以标准的I/O设备（通常为终端设备）为输入输出对象，如：（各系统均提供）
putchar、getchar、printf、scanf、puts、gets

数据输入输出在C语言中的实现 (2)



- 在使用**C**语言库函数时，须用预编译命令“**#include**”将所需要的“头文件”包括到用户源文件中。
 - 头文件 —— 包含了将用到的库函数的有关信息，扩展名为“**.h**”。
- **[例] stdio.h**即标准输入输出函数库，包含了与标准**I/O**库有关的变量定义、宏定义和函数原型（函数首部）等。
 - 调用标准**I/O**库函数时，文件开头应有如下预编译命令：
#include <stdio.h>
 //在系统指定的**include**目录中找（标准方式）
或 **#include "stdio.h"**
 //先在用户的当前目录下找，再按标准方式
 - 有些编译器（如**TC2**）允许在仅使用**printf**和**scanf**时，省略**#include**命令。



用printf函数输出数据

printf函数



【作用】 向终端（/系统隐含指定的输出设备）输出若干个任意类型的数据。

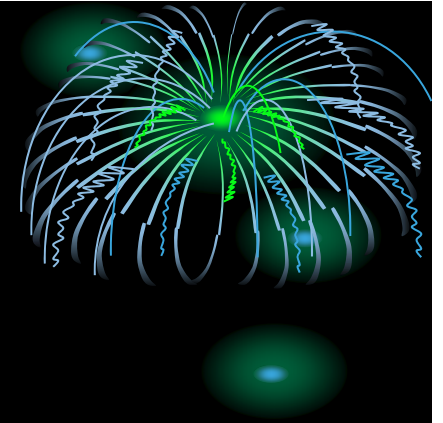
【说明】 **#include <stdio.h>**

int printf (const char *format, ...);

- **格式控制字符串 / 转换控制字符串**，包括两种信息：
 - 1. 格式说明：**由“%”和**格式字符**组成，作用是将输出的数据转换为指定的格式输出。
 - 想输出字符“%”，应在“格式控制”字符串中用连续的两个%表示！
 - 2. 普通字符：**即要**原样输出**的字符。
- **输出表列：**需要输出的一些数据，可以是常量、变量或表达式。如：**printf(“%d, %c\n”, i, c);**

【返回值】 成功时返回输出的字节数，否则返回**EOF**。 7

例子



printf函数的一般格式

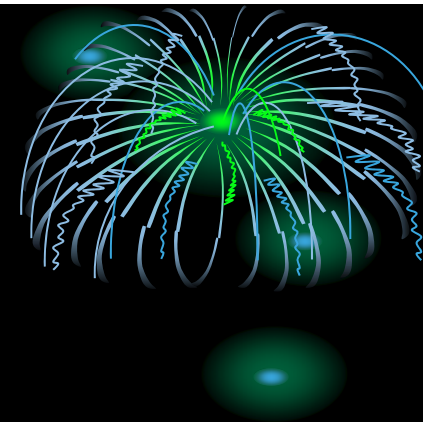
printf（格式控制，输出表列）

如：

```
printf("i=%d,c=%c\n",i,33);
```

格式说明

例子



printf函数的一般格式

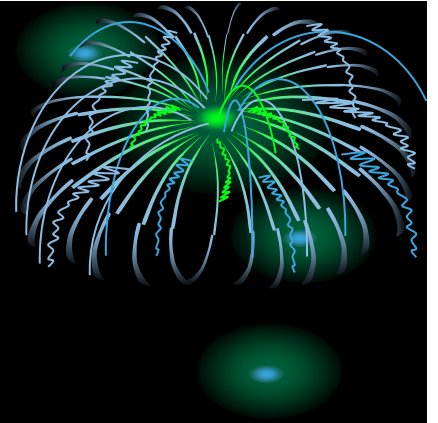
printf（格式控制，输出表列）

如：

```
printf("i=%d,c=%c\n",i,33);
```

普通字符

例子



printf函数的一般格式

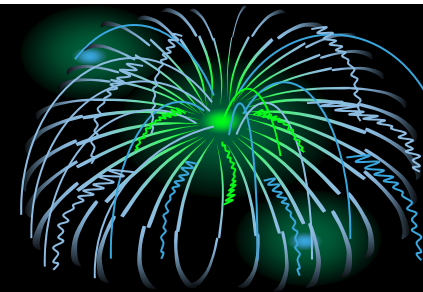
printf（格式控制，输出表列）

如：

```
printf("i=%d,c=%c\n",i,33);
```

可以是常量、变量或表达式

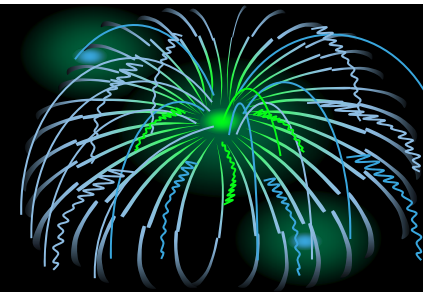
与格式相关的几个基本概念



- **数据宽度**: 一个数据在输入/输出时所占用的字符数。
- **对齐方式**: 输出时, 需要考虑类似表格中各列的对齐问题。(C通常默认右对齐)
 - **右对齐**: 通常为数值数据所采用。
 - **左对齐**: 通常为非数值数据所采用。

书名	作者	价格
C程序设计	谭浩强	26.00
儿童画报	张三	4.50
辞海	××出版社	240.00

printf函数 (2)



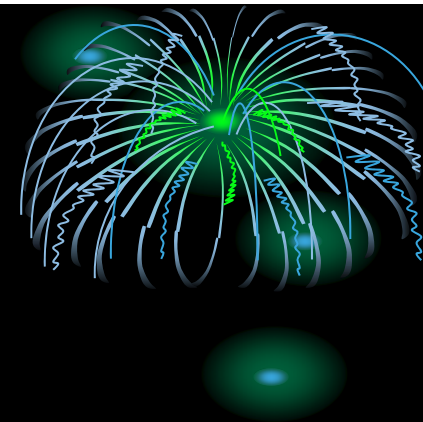
- 格式字符**d**、**i** —— 十进制整数

形如: **%** **[-]** **[m]** **[-n]** **[l|ll]** **d|i**

【说明】

- **-**表示左对齐，省略则为右对齐。
- **m**指定了输出的最小字符（含空格）宽度。若输出位数小于**m**，则在左端/右端（决定于对齐方式）用空格填充；若大于**m**，则按实际位数输出。
- **n**指定了最小输出数位（含符号位，但不含空格）。若实际位数小于**n**，则左端用**0**填充；若大于**n**，则按实际位数。
- **l**表示按长整型输出，**ll**表示按双长整型输出
 - ❖注意：此时系统将直接从对应参数处读取相应字节的数据，因此若相应数据为基本整数，输出的数值将是错误的，故小的整型常量以此格式输出时要在后面加上**L**，基本整变量则应进行强制类型转换！

printf函数 (3)



- 格式字符**c** —— 字符

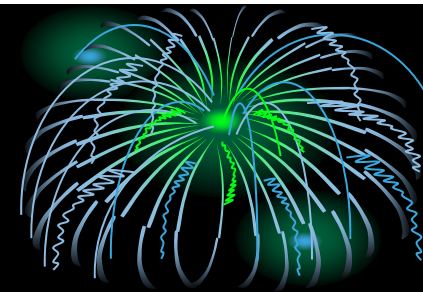
形如: **%** **[—]** **[m]** **c**

[说明] (—、**m**同**d**)

- —表示左对齐，省略则为右对齐。
- **m**指定了输出的最小字符（含空格）宽度。若输出位数小于**m**，则在左端/右端（决定于对齐方式）用空格填充；若大于**m**（仅当**m=0**时），则按一个字符输出。
- 整型数据也可用**%c**输出，不过会自动转换为**0~255**范围内的值（即**char**型）；反之，字符数据也可用整数形式输出。

[例E4 X3]

printf函数 (4)



- 格式字符**s** —— 字符串

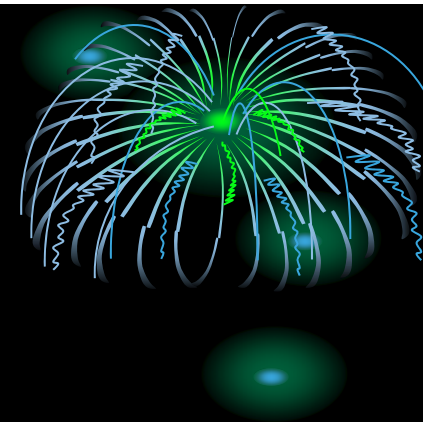
形如: **% [-] [m] [.n] s**

[说明] (-、**m**同**d**)

- -表示左对齐, 省略则为右对齐。
- **m**指定了输出的最小字符(含空格)宽度。若输出位数小于**m**, 则在左端/右端(决定于对齐方式)用空格填充; 若大于**m**, 则按实际输出字符数输出; 即总宽度为**max(m, min(n, strlen(s)))**。缺省取**min(n, strlen(s))**。
- **n**指定了输出的最大字符数。若实际字符数小于**n**, 则按实际字符数输出; 若大于**n**, 则截取前面**n**个字符; 即实际输出字符数为**min(n, strlen(s))**。缺省取**strlen(s)**。

[例E4 X4]

printf函数 (5)



- 格式字符**f** —— 小数形式

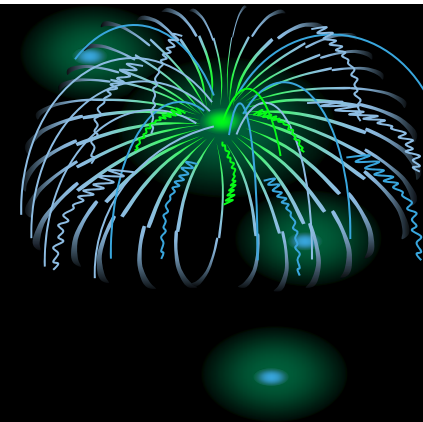
形如: **% [-] [m] [-.n] [L]f**

[说明]

- -表示左对齐，省略则为右对齐。
- **m**指定输出的最小宽度（符号和小数点各占一列）。若实际输出位数大于**m**，则按实际输出位数输出。
- **n**指定了小数点后的位数。缺省**n=6**。
- **L**用于输出**long double**型数据
- 单精度实数的有效数位一般为**7**位，双精度实数的一般为**16**位。

[例E4_X5]

printf函数 (6) *



- 格式字符**e** —— 指数形式

形如: % **[—] [m] [.n] e|E**

[说明] (—、**m**同**f**)

- **n**指定的是数据的尾数部分的数字位数。缺省**n=6**。
- 指数部分至少占**4**位（不足用**0**补），其中“**e**”和指数符号各占**1**位。

[例E4_X6]

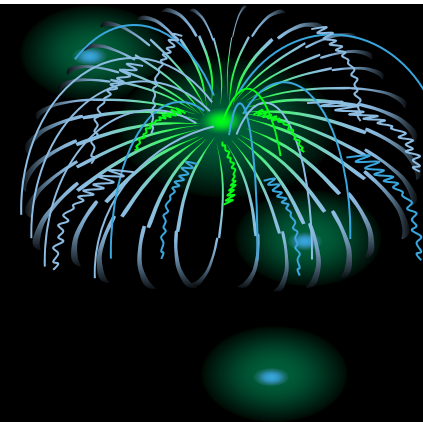
- 格式字符**g** —— 实数

形如: % **g|G**

[说明] 根据数值的大小，自动选**f**或**e**格式中输出占宽度较小者，并不输出无意义的**0**。

— 书上**P80~81**的输出有误! **[例E4_X6]**

printf函数 (7)



- **格式字符o** —— 八进制整数
形如: % [-] [m] [.n] [l] o
【说明】（-、n、m、l的说明同d）
 - 以补码形式输出，因此输出的数值不再有符号。
- **格式字符x** —— 十六进制整数
形如: % [-] [m] [.n] [l] x|X
【说明】（-、n、m、l的说明同d）
 - 用x则字母以小写输出，用X则以大写输出。
 - 以补码形式输出，同样不再有符号。
- **格式字符u** —— 十进制无符号整数
形如: % [-] [m] [.n] [l] u
【说明】（-、n、m、l的说明同d）

[例E4 X2]

小结：格式声明的一般形式

% 附加字符 格式字符

• P73 表3.6

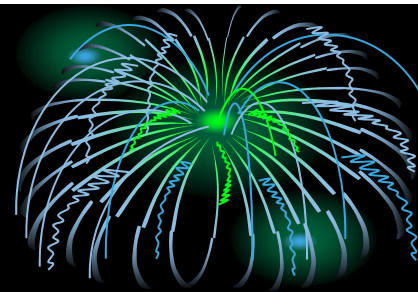


表 3.6 printf 函数中用到的格式字符

格式字符	说 明
d,i	以带符号的十进制形式输出整数(正数不输出符号)
o	以八进制无符号形式输出整数(不输出前导符 0)
x, X	以十六进制无符号形式输出整数(不输出前导符 0x), 用 x 则输出十六进制数的 a~f 时以小写形式输出。用 X 时, 则以大写字母输出
u	以无符号十进制形式输出整数
c	以字符形式输出, 只输出一个字符
s	输出字符串
f	以小数形式输出单、双精度数, 隐含输出 6 位小数
e, E	以指数形式输出实数, 用 e 时指数以“e”表示(如 1.2e+02), 用 E 时指数以“E”表示(如 1.2E+02)
g, G	选用%f或%e格式中输出宽度较短的一种格式, 不输出无意义的 0。用 G 时, 若以指数形式输出, 则指数以大写表示

小结 (2)

• P74 表3.7 (修饰符/附加符号)

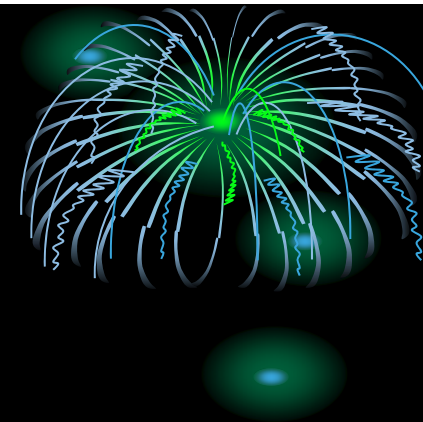
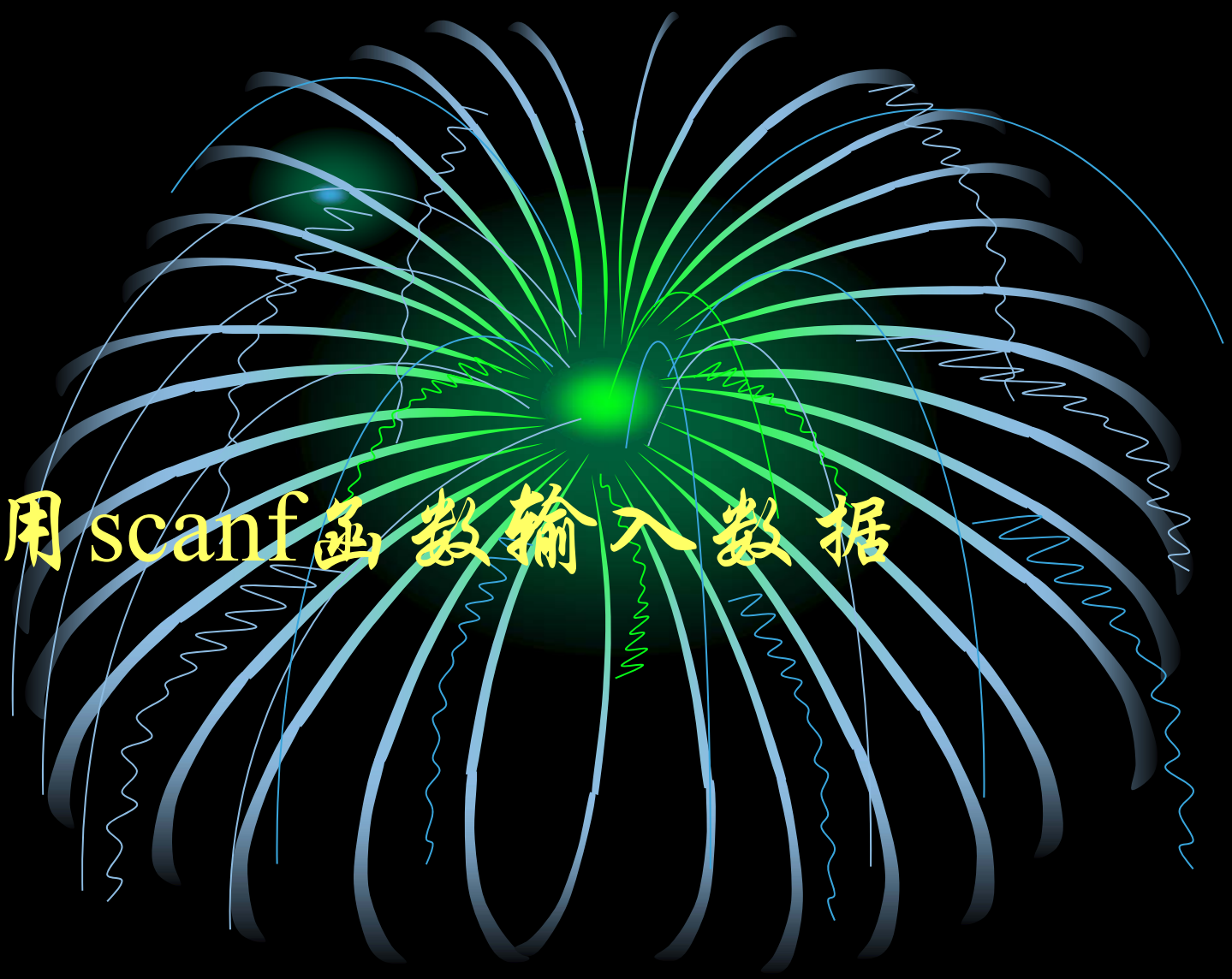


表 3.7 printf 函数中用到的格式附加字符

字 符	说 明
l	用于长整型整数,可加在格式符 d、o、x、u 前面
m(代表一个正整数),.	数据最小宽度
n(代表一个正整数)	对实数,表示输出 n 位小数;对字符串,表示截取的字符个数
—	输出的数字或字符在域内向左靠



用scanf函数输入数据

scanf函数



【作用】 将从终端（/系统隐含指定的输出设备）输入的若干个任意类型的数据保存到相应的变量中。

【说明】 **#include <stdio.h>**
 int scanf (const char *format, ...);

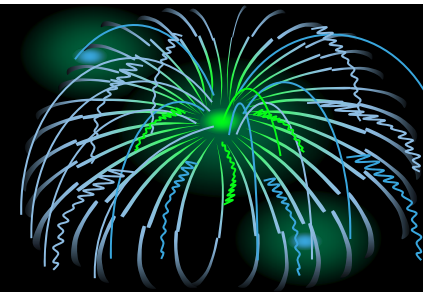
- 格式控制字符串 / 转换控制字符串（基本同**printf**）
- 地址表列：由若干个用于存放输入数据的地址组成，可以是变量的地址，或字符串的首地址。

【返回值】 成功时返回输入的数据个数。

如：**scanf(“%d%d%d”,&a,&b,&c);**

- 此时，输入的数据之间可用空格/回车键/**tab**键作为间隔，而不能使用其它符号。（行缓冲）

scanf函数 (2)



- 格式字符**d**、**i**、**u**、**x**、**c**、**s**、**f**、**e**、**g**

形如: **% [*] [m] [h|l] <格式字符>**

【说明】

- **m** 指定输入数据所占的宽度（列数），应为正整数；
- **l|L** 用于输入**long**型及**double**型数据；
h 用于输入短整型数据。
- ***** 表示本输入项在读入后不赋给相应变量。
- **f**、**e**、**g**的作用相同；格式字符大/小写作用均相同。

P76 表3.8

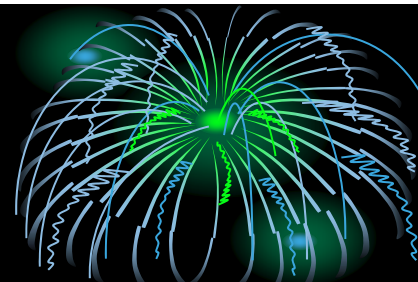


表 3.8 scanf 函数中所用到的格式字符

格式字符	说 明
d,i	用来输入有符号的十进制整数
u	用来输入无符号的十进制整数
o	用来输入无符号的八进制整数
x, X	用来输入无符号的十六进制整数(大小写作用相同)
c	用来输入单个字符
s	用来输入字符串,将字符串送到一个字符数组中,在输入时以非空白字符开始,以第一个空白字符结束。字符串以串结束标志'\0'作为其最后一个字符
f	用来输入实数,可以用小数形式或指数形式输入
e, E, g, G	与 f 作用相同,e 与 f、g 可以互相替换(大小写作用相同)

P76 表3.9

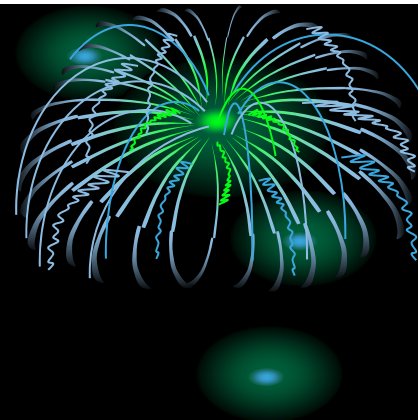
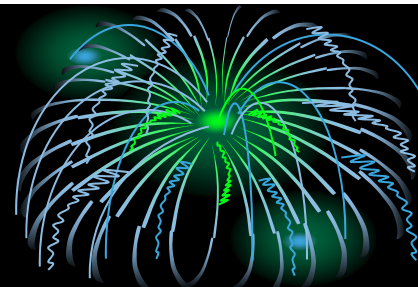


表 3.9 scanf 函数中用到的格式附加字符

字符	说 明
l	用于输入长整型数据(可用%ld,%lo,%lx,%lu)以及 double 型数据(用%lf 或%le)
h	用于输入短整型数据(可用%hd,%ho,%hx)
域宽	指定输入数据所占宽度(列数),域宽应为正整数
*	表示本输入项在读入后不赋给相应的变量

scanf函数 (3)



【注意事项】

- 要将输入数据存放在变量中，必须用取地址符“&”取得其地址；

错例: `scanf("%f%f%f",a,b,c);`

正确: `scanf("%f%f%f",&a,&b,&c);`

- 如果在“格式控制”中除了格式说明符外还有其它字符，则在输入时应该在对应位置上输入与这些字符相同的字符。

如: `scanf("a=%f,b=%f,c=%f",&a,&b,&c);`

`1 3 2✓`

错

`a=1,b=3,c=2✓`

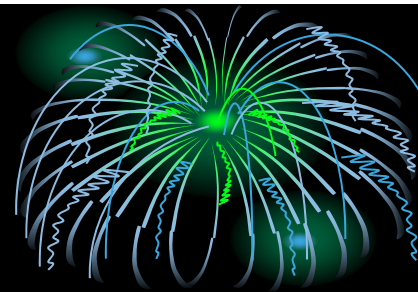
对

`a=1 b=3 c=2✓`

错

- 若是n个空格，则应输入m ($m \geq n$) 个空格。

scanf函数 (4)



【注意事项】

- 使用**%c**输入时，空格等键盘上可键入的字符都作为有效字符（转义字符无需转义！）。

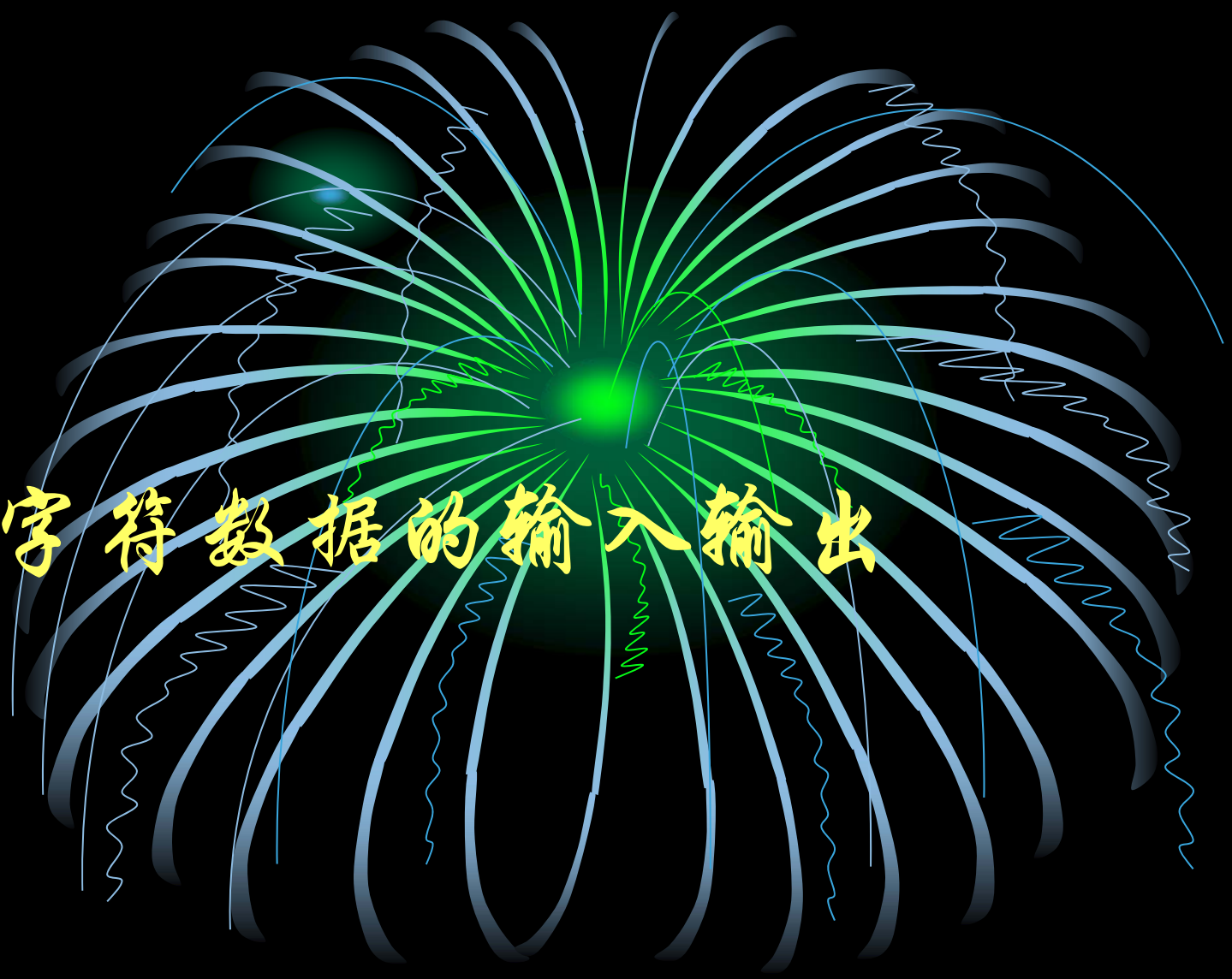
如：**scanf("%c%c%c",&c1,&c2,&c3);**

- 中间不要添加空格！
- 在输入数据时，当遇到如下情况之一时，该数据认为结束：
 - ①遇到空字符（空格，回车，**Tab**）；
 - ②按指定的宽度读取结束；如：**scanf("%3d%2d",&a,&b);**
 - ③遇到非法输入（相对于格式符所限定的类型和格式而言）。

如：**scanf("%d%c%f",&a,&b,&c);**

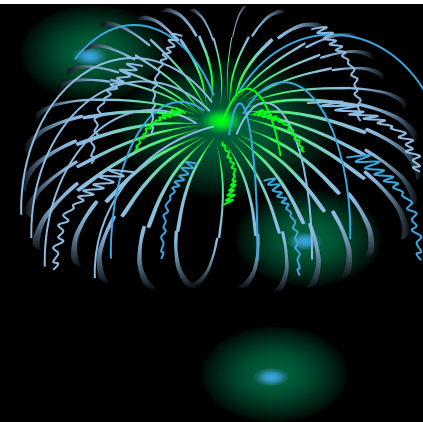
输入**1234a123o.26**，则**a**值为**1234**，**b**值为**'a'**，**c**值为**123**

26



字符数据的输入输出

putchar函数



【说明】 **#include <stdio.h>**
 int putchar (int ch);

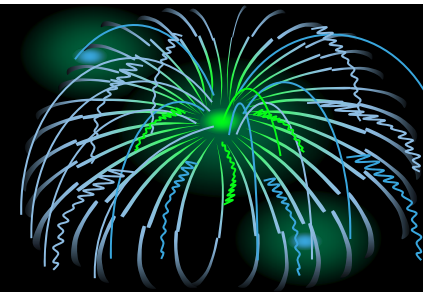
【作用】 向终端（/系统隐含指定的输出设备）
 输出一个字符。

【返回值】 成功时返回**ch**的值，否则返回**EOF**
 (-1)。

【注意】

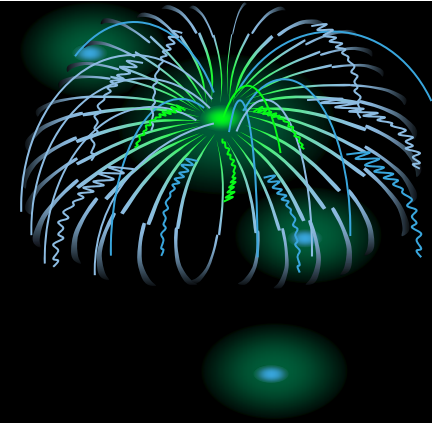
- 实参可以是字符型或者整型；当提供整型参数时，当成**ASCII**码值。
- 既可输出字符，也可输出控制字符（用**转义符**）

【例3.8】 先后输出BOY三个字符。



【解题思路】

- 定义**3**个字符变量，分别赋以初值**B**、**O**、**Y**
- 用**putchar**函数输出这**3**个字符变量的值



```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    char a='B',b='O',c='Y';
```

```
    putchar(a);           //向显示器输出字符B
```

```
    putchar(b);
```

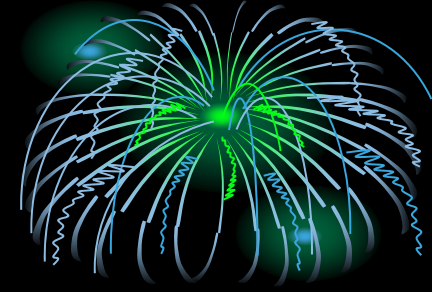
```
    putchar(c);
```

```
    putchar ('\n');      //向显示器输出换行符
```

```
    return 0;
```

```
}
```

BOY



```
#include <stdio.h>
```

```
int main ( )
```

改为 `int a=66,b=79,c=89;`

```
{
```

```
    char a='B',b='O',c='Y';
```

```
    putchar(a);
```

```
    putchar(b);
```

```
    putchar(c);
```

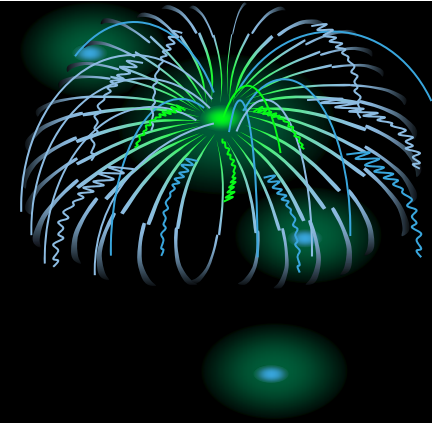
```
    putchar ('\n');
```

```
    return 0;
```

```
}
```

BOY

例子



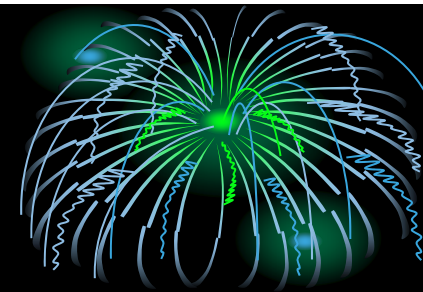
putchar('\101') (输出字符 A)

putchar('\n') (输出单撇号字符')

putchar('\015')
(输出回车, 即光标移到本行开头)



getchar函数



【说明】 **#include <stdio.h>**
 int getchar (void);

【作用】 从终端（/系统隐含指定的输入设备）输入一个字符。

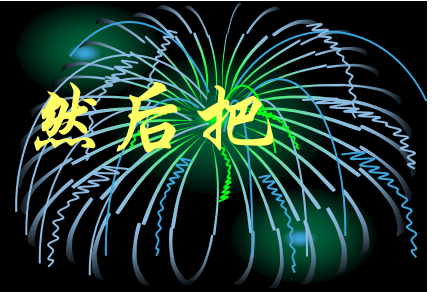
【返回值】 成功时返回所读入的字符，否则（失败或读到文件尾）返回**EOF**。

【注意】

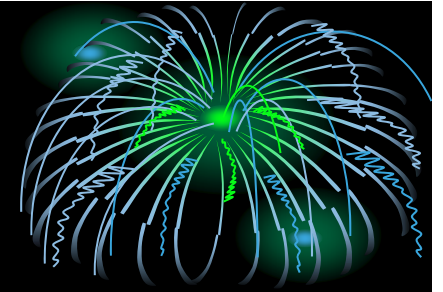
- 必须输入回车键，字符才能被接收。（行缓冲）
- 只能接收一个字符，多余的字符被忽略（会被后继的**getchar**读取）。
- 返回值可赋给一个字符变量/整型变量

【例E4 2】、【例E4 z2】

【例3.9】 从键盘输入BOY三个字符，然后把它们输出到屏幕。



- 解题思路：
 - 用3个**getchar**函数先后从键盘向计算机输入**BOY**三个字符；
 - 用**putchar**函数输出



```
#include <stdio.h>
```

```
int main ( )
```

```
{ char a,b,c;
```

```
  a=getchar();
```

```
  b=getchar();
```

```
  c=getchar();
```

```
  putchar(a); putchar(b); putchar(c);
```

```
  putchar('\n');
```

```
  return 0;
```

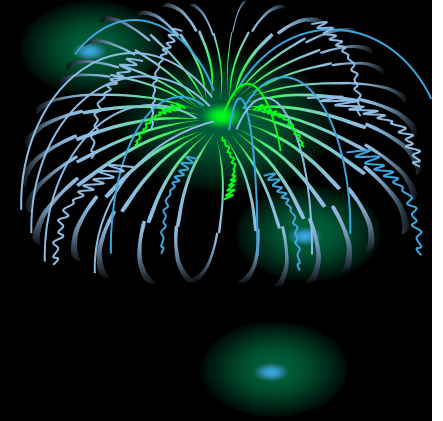
```
}
```

不要在输入每个字符后马上按回车，否则回车也会作为一个字符输入。

```
BOY
BOY
```

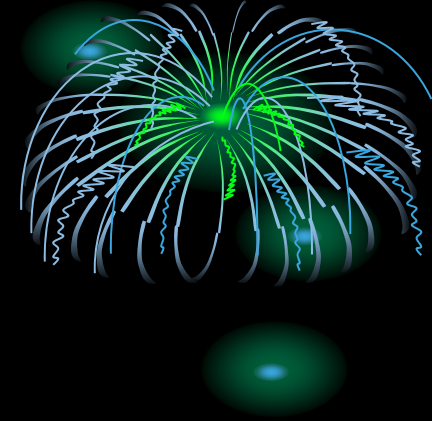
```
B
O
B
O
```


改写



```
#include <stdio.h>
int main ( )
{ char a,b,c;
  a=getchar(); putchar(getchar());
  b=getchar();
  c=getchar();
  putchar(a); putchar(b); putchar(c);
  putchar('\n');
  return 0;
}
```

改写



```
#include <stdio.h>
```

```
int main ( )
```

```
{ char a,b,c;
```

```
    b=getchar();
```

```
    c=getchar();
```

```
    putchar('\n');
```

```
    return 0;
```

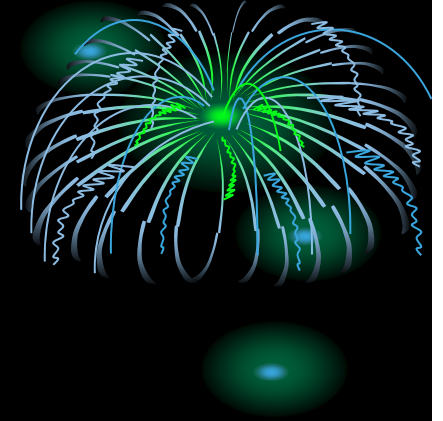
```
}
```

```
    putchar(getchar());
```

```
    putchar(getchar());
```

```
    putchar(b); putchar(c);
```

改写



```
#include <stdio.h>
```

```
int main ( )
```

```
{ char a,b,c;
```

```
    c=getchar();
```

```
    putchar('\n');
```

```
    return 0;
```

```
}
```

```
    putchar(getchar());
```

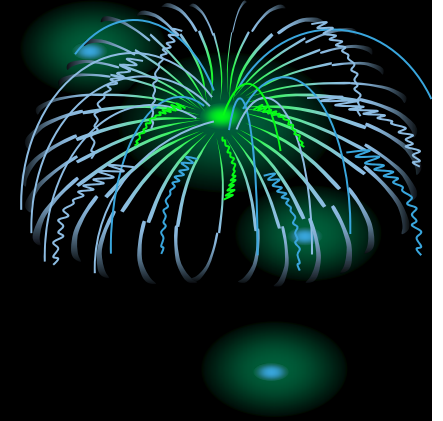
```
    putchar(getchar());
```

```
    putchar(getchar());
```

```
    putchar(c);
```



改写



```
#include <stdio.h>
```

```
int main ( )
```

```
{ char a,b,c;
```

```
    putchar(getchar());
```

```
    putchar(getchar());
```

```
    putchar(getchar());
```

```
    putchar('\n');
```

```
    return 0;
```

```
}
```

```
BOY
```

```
BOY
```

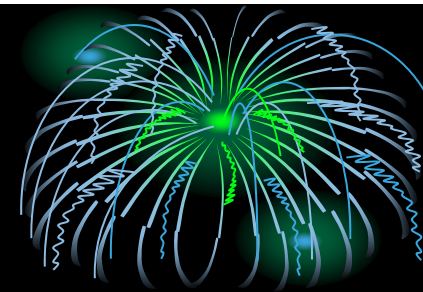
【例3.10】 改写例3.3程序，使之可以适用于任何大写字母。从键盘输入一个大写字母，在屏幕上显示对应的小写字母。

例3.3程序

【解题思路】

- 用**getchar**函数从键盘上读入一个大写字母；
- 把它转换成小写字母；
- 用**putchar**函数输出该小写字母。

```
#include <stdio.h>
int main ( )
{
    char c1,c2;
    c1='A';
    c2=c1+32;
    printf("%c\n",c2);
    printf("%d\n",c2);
    return 0;
}
```



```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    char c1,c2;
```

```
    c1 = getchar();
```

```
    c2 = c1 + 32;
```

//从键盘读入一个大写字母,赋给字符变量 c1

//求对应小写字母的 ASCII 代码,放在字符变量 c2 中

```
printf("大写字母: %c\n 小写字母: %c\n",c1,c2);
```

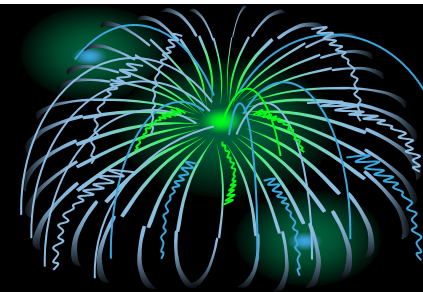
```
    return 0;
```

```
}
```

【bug】 未对输入字符是否为
大写字母进行判断（需用到下
一章的**if**语句）！

增加输出结果的可读性

getch函数*



【说明】 **#include <stdio.h>**
 int getch (void);

【作用】 （同**getchar**）从终端（/系统隐含指定的输入设备）输入一个字符。

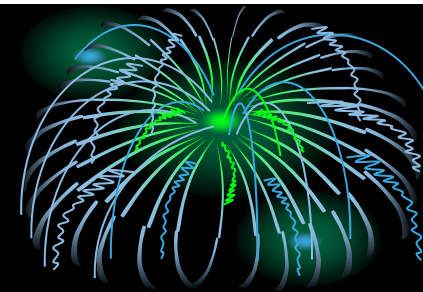
【返回值】 （同**getchar**）成功时返回所读入的字符，否则（失败或读到文件尾）返回**EOF**。

【注意】

- 没有行缓冲的，接收到一个字符后立即返回之。
- 不带有回显。

【例E4_2'】

getche 函数*



【说明】 **#include <stdio.h>**
 int getche (void);

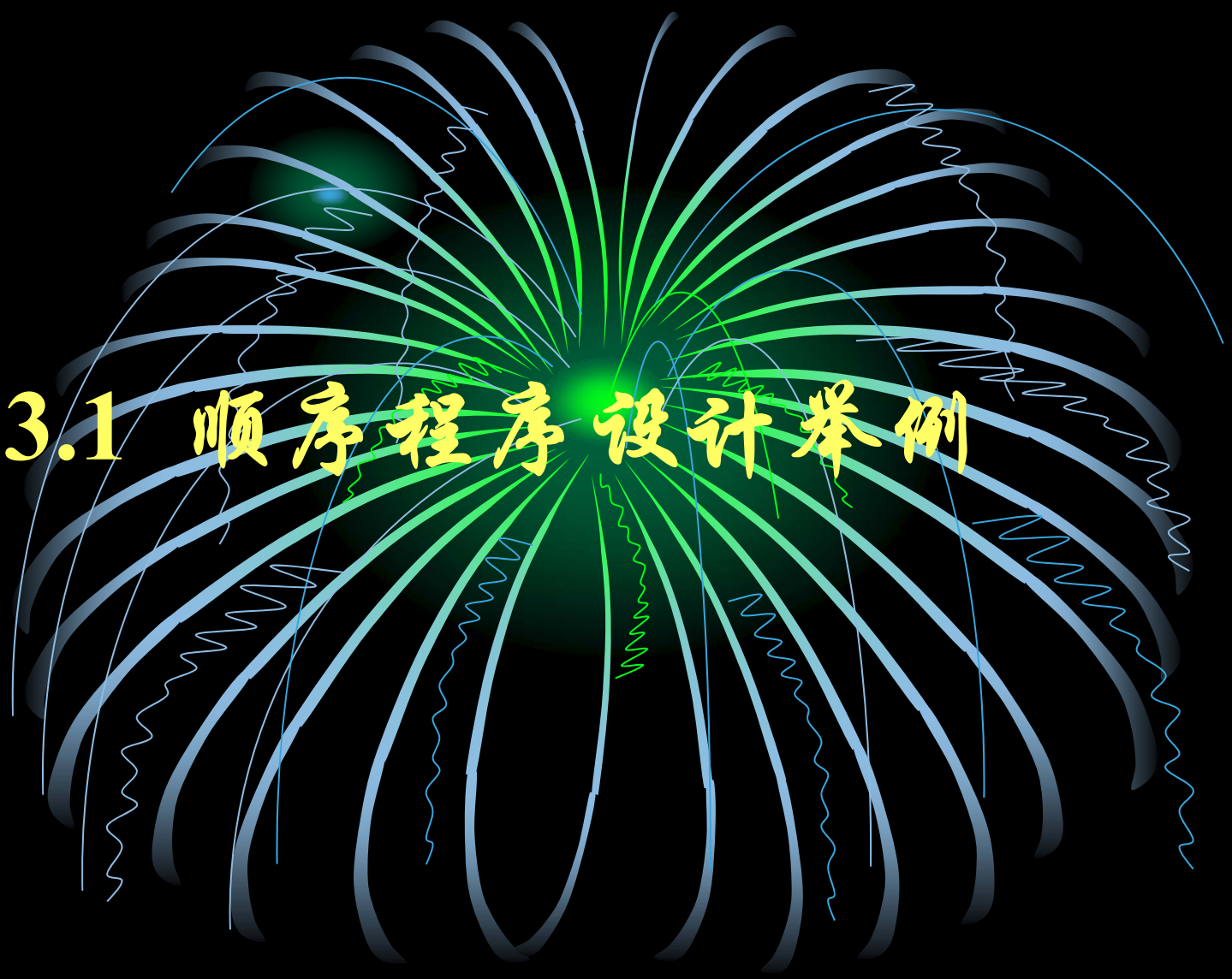
【作用】 （同**getchar**）从终端（/系统隐含指定的输入设备）输入一个字符。

【返回值】 （同**getchar**）成功时返回所读入的字符，否则（失败或读到文件尾）返回**EOF**。

【注意】

- 没有行缓冲的，接收到一个字符后立即返回之。（同**getch**）
- 有回显。

【例E4_2”】



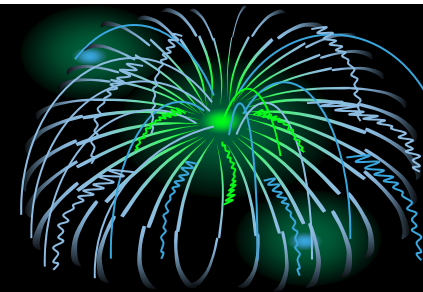
§ 3.1 顺序程序设计举例

编写C语言程序应具备的知识和能力

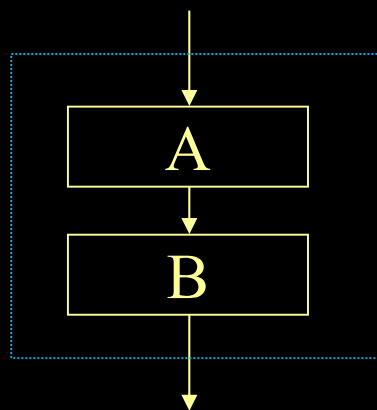


1. 要有正确的解题思路，即学会设计算法；
2. 掌握C语言的语法，知道怎样使用C语言所提供的功能编写出一个完整的、正确的程序；
3. 在设计算法和编写程序时，采用结构化程序设计方法。

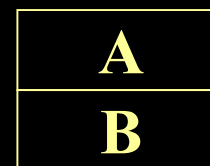
三种基本结构及其流程图描述



(1) 顺序结构



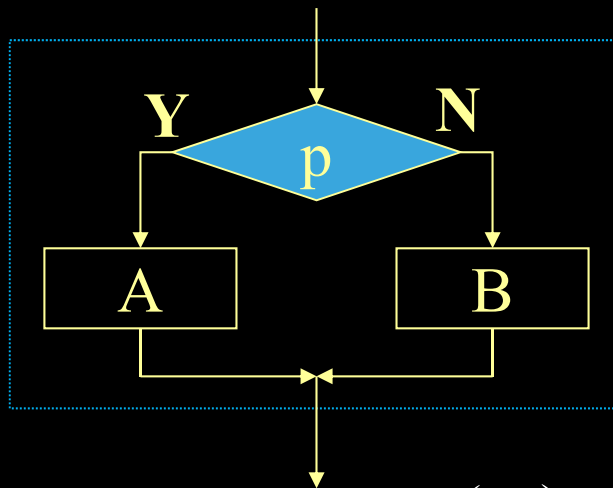
(a) 传统流程图表示



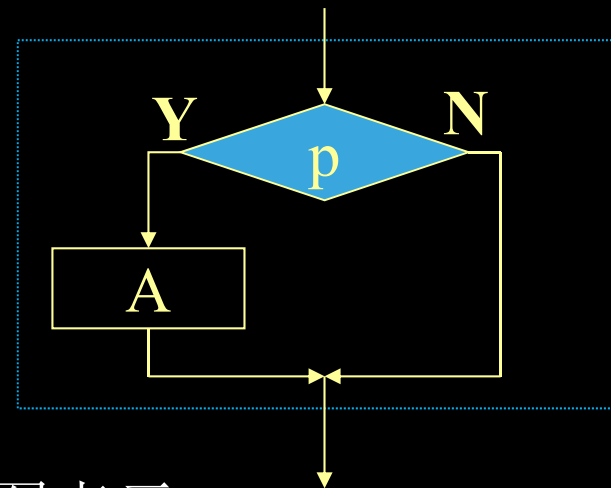
(b) **N-S**流程图表示

三种基本结构及其流程图描述 (2)

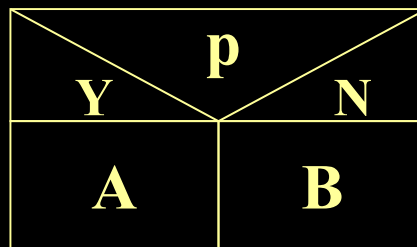
(2) 选择 (I选取/分支) 结构



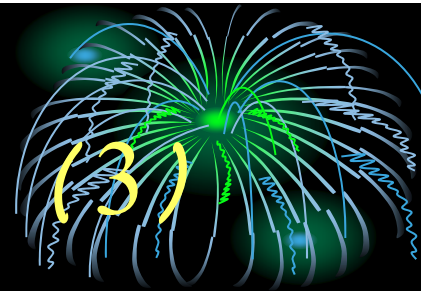
(a) 传统流程图表示



(b) N-S流程图表示

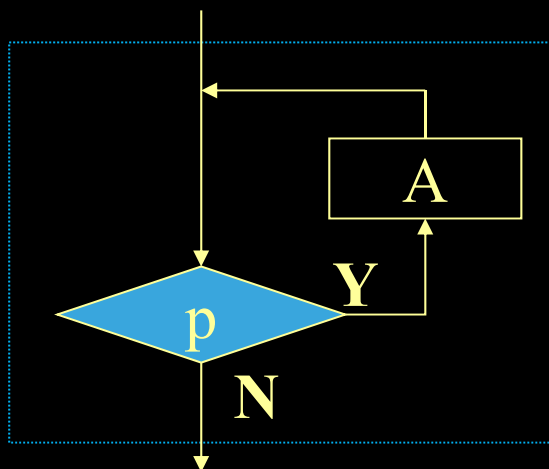


三种基本结构及其流程图描述 (3)

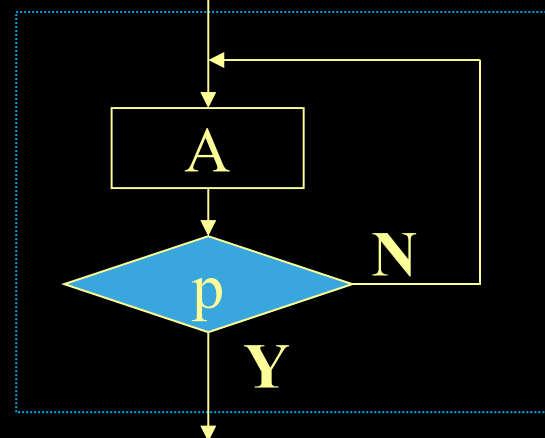


(3) 循环 (I重复) 结构

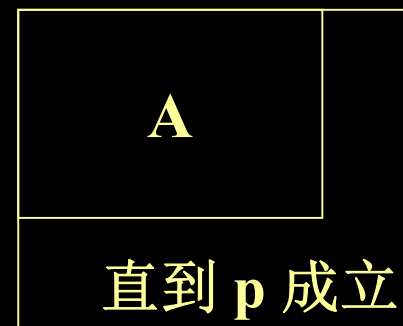
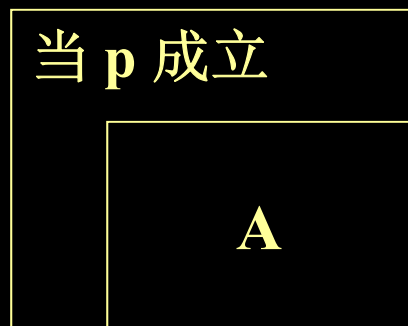
当型 (While型)



直到型 (Until型)

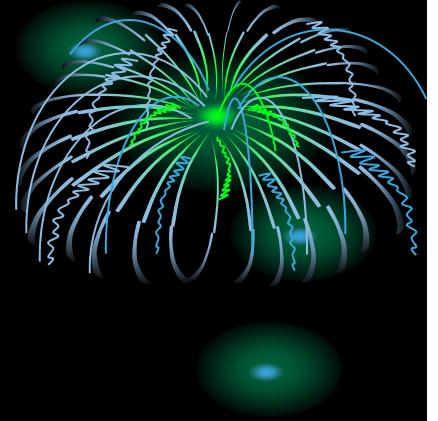


(a) 传统流程图表示



(b) N-S流程图表示

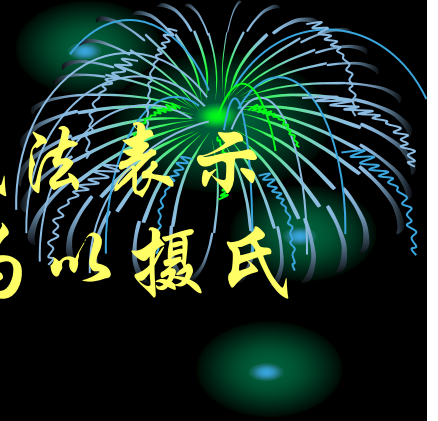
三种基本结构的共同特点



- (1)** 只有一个入口
- (2)** 只有一个出口
- (3)** 结构内的每一部分都有机会被执行到。也就是说，对每一个框来说，都应当有一条从入口到出口的路径通过它
- (4)** 结构内不存在“死循环”
 - 不存在无规律的转向，只在结构内部允许分支和跳转。
 - 可根据需要构造其它满足上述特点的基本结构（如：多分支选择结构）
- 用上述三种基本结构组成的算法结构，可以解决任何复杂的问题。



【例3.1】 有人用温度计测量出用华氏法表示的温度（如64 F），今要求把它转换为以摄氏法表示的温度（如17.8 C）。

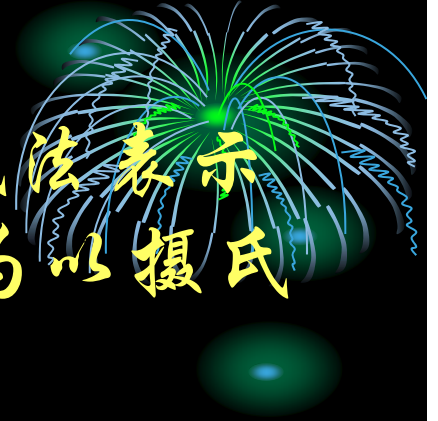


【解题思路】 找到二者间的转换公式

$$c = \frac{5}{9}(f - 32)$$

f代表华氏温度，c代表摄氏温度

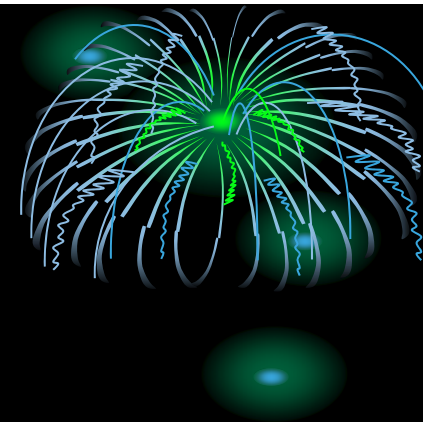
【例3.1】 有人用温度计测量出用华氏法表示的温度（如64 F），今要求把它转换为以摄氏法表示的温度（如17.8 C）。



【算法】N-S图

输入 f 的值
$c = \frac{5}{9}(f - 32)$
输出 c 的值

【例3.1】程序



```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    float f,c;    定义f和c为单精度浮点型变量
```

```
    f=64.0;      指定f的值
```

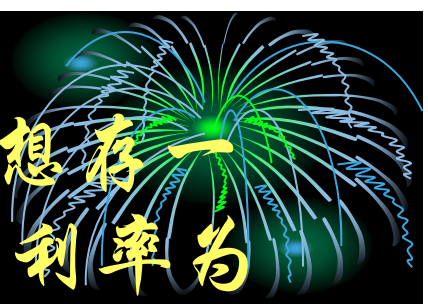
```
    c=(5.0/9)*(f-32);    计算c的值
```

```
    printf("f=%f\nc=%f\n",f,c); 输出f和c的值
```

```
    return 0;
```

```
}
```

```
f = 64.000000  
c = 17.777778
```

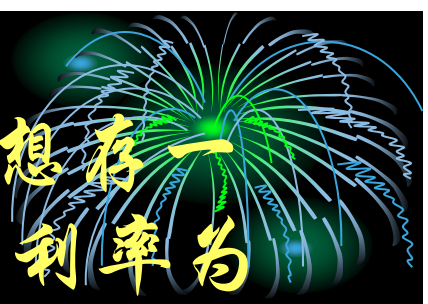


【例3.2】计算存款利息。有1000元，想存一年。有三种方法可选：（1）活期，年利率为 r_1 ；（2）一年期定期，年利率为 r_2 ；（3）存两次半年定期，年利率为 r_3 。请分别计算出一年后按三种方法所得到的本息和。

【解题思路】确定计算本息和的公式。
若存款额为 p_0 ，则：

- 活期存款一年后本息和为 $p_1 = p_0(1 + r_1)$
- 一年期定期存款，一年后本息和为 $p_2 = p_0(1 + r_2)$
- 两次半年定期存款，一年后本息和为

$$p_3 = p_0 \left(1 + \frac{r_3}{2}\right) \left(1 + \frac{r_3}{2}\right)$$

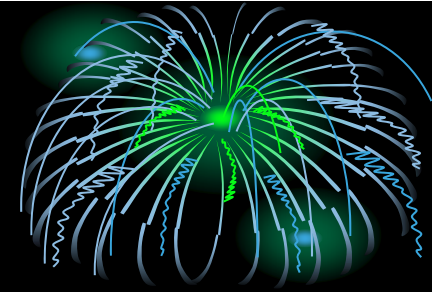


【例3.2】计算存款利息。有1000元，想存一年。有三种方法可选：（1）活期，年利率为 r_1 ；（2）一年期定期，年利率为 r_2 ；（3）存两次半年定期，年利率为 r_3 。请分别计算出一年后按三种方法所得到的本息和。

【算法】N-S图：

输入 p_0, r_1, r_2, r_3 的值
计算 $p_1 = p_0(1 + r_1)$
计算 $p_2 = p_0(1 + r_2)$
计算 $p_3 = p_0(1 + \frac{r_3}{2})(1 + \frac{r_3}{2})$
输出 p_1, p_2, p_3

【例3.2】程序



```
#include <stdio.h>
```

```
int main ( )
```

```
{float p0=1000, r1=0.0036, r2=0.0225,  
      r3=0.0198, p1, p2, p3;
```

```
  p1 = p0 * (1 + r1);
```

```
  p2 = p0 * (1 + r2);
```

```
  p3 = p0 * (1 + r3/2) * (1 + r3/2);
```

```
  printf("%f\n%f\n%f\n", p1, p2, p3);
```

```
  return 0;
```

```
}
```

定义变量同时赋予初值

```
1003.599976
```

```
1022.500000
```

```
1019.898010
```

